GPU Programming(2026) Assignment 4: Wildfire Simulation on GPUs

Department of Computer Science & Engineering, IIT Madras

Due: 7 November 2026, 23:59 IST | Marks: 16

Assignment Submission Guidelines

1. You are given `harness.cu`, `wildfire.h`, a skeleton `wildfire.cu`, the `run_testcases.sh` and `job.sh` scripts, the converter `convert.py` (Section 6), and the public test cases in the `input` and `output` folders.
2. Fill your code in the `wildfire.cu` file by implementing the `run_simulation()` function. Its full contract is described in `wildfire.h`. You are free to use any number of kernels and helper functions.
3. Do not modify `harness.cu` or `wildfire.h`. The harness reads the input, warms up the GPU, calls your `run_simulation()`, and writes the output. Only that call is timed, including any host–device copies inside it; reading and writing the files is not. Do not write any print statements.
4. Compile it with: `nvcc -arch=sm_70 harness.cu wildfire.cu -o main`
5. Run the `run_testcases.sh` script to test your code on all the public test cases given in the `input` and `output` folders. It needs a machine with an NVIDIA GPU. If you have access to the Aqua cluster, you can instead submit the provided `job.sh` script with `qsub`; it compiles your code and runs `run_testcases.sh`.
6. Submit your `wildfire.cu` as `ROLL_NO_ALL_CAPS.cu` on Moodle. Your file must compile with the command above and pass the test cases as it is, with no manual edits by the evaluator.
7. Note: You can use the STL and the Thrust library for this assignment.

---

1 Problem Overview

Simulations are a big part of GPU programming. In this assignment, we will simulate a wildfire, with firefighters trying to put it out.

The fire burns on a large terrain grid (up to 8192×8192 cells), where each cell carries its fuel, elevation and state. The fire spreads through a 5×5 neighbourhood: burning cells heat the cells around them, with the heat depending on wind and slope, and cells that receive enough heat ignite. Burning cells consume their fuel and burn out when it runs out. A cell with no fuel can never be set on fire.

The firefighters work in crews that move across the grid, always heading for the closest fire (by the number of steps needed to reach it). Each crew can put out a limited number of fires per tank load, and once its tank is empty, it must return to the closest fire station to refill. A cell that has been put out becomes wet, so it can never be set on fire again.

2 The Terrain Grid

The terrain is a square grid of $N \times N$ cells, with $N \leq 8192$. Each cell has three properties: its **fuel**, its **elevation**, and its **state**.

- **Fuel** is an integer from 0 to 255: the amount of burnable material left in the cell. A cell may start with 0 fuel, and a cell with 0 fuel can never be set on fire. A burning cell loses fuel every step.
- **Elevation** is an integer from 0 to 255: the height of the cell, used to compute the slope between neighbouring cells.
- **State** is one of:
  - *unburnt*: not yet on fire;
  - *burning*: on fire and consuming fuel;
  - *burnt*: its fuel ran out while burning;
  - *wet*: put out by a firefighter.

  Wet and burnt cells can never be set on fire again.

At the start of the simulation, some cells are already on fire; these cells are specified in the input. All the other cells start off *unburnt*.

There are also a few **fire stations** scattered across the terrain, whose locations are given in the input. Fire stations can never be set on fire. The grid may give them values for fuel and elevation, but those values are ignored.

Firefighter crews also stand on cells of the grid, and their initial positions are given in the input. Any number of crews may stand on the same cell, and a crew can stand on or move through a cell in any state, including a burning one. A crew does not change the cell it stands on: the cell can still catch fire, burn and give heat as usual.

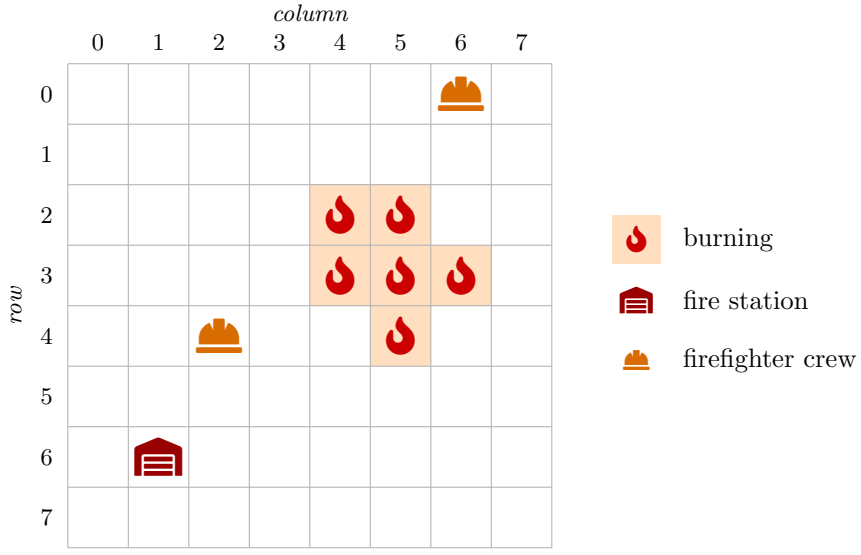


Figure 1: An 8×8 terrain.

3 Fire Spread and Burning

Fire spreads through a 5×5 neighbourhood. In every step, each burning cell gives heat to every other cell in the 5×5 square centred on it (cells up to 2 rows and 2 columns away). Cells outside the grid are ignored. The heat a cell receives from one burning neighbour depends on the wind and on the slope between the two cells.

Parameters

The following integer parameters are given in the input:

- H: the **base heat**. Every burning cell gives H units of heat to each cell in its 5×5 square, before the wind and slope effects below are applied.
- (d_r, d_c): the **wind direction**, with $d_r, d_c \in \{-1, 0, 1\}$, not both 0. The wind blows from each cell (r, c) towards the cell $(r + d_r, c + d_c)$, so it is one of the 8 directions. For example, $(0, 1)$ blows towards higher column numbers, $(-1, 0)$ towards lower row numbers, and $(1, 1)$ diagonally, towards higher row and column numbers. The wind is the same across the whole grid and does not change over time.
- $W \geq 1$: the **wind factor**. A cell that lies straight downwind of a burning cell, 1 or 2 cells away along the wind direction, receives W times the heat from it. A cell that lies straight upwind of it, 1 or 2 cells away against the wind direction, receives the heat divided by W.
- $S \geq 1$: the **slope factor**. Fire spreads more easily uphill. A cell that is higher than a burning cell by more than D receives S times the heat from it, and a cell that is lower by more than D receives the heat divided by S.
- $D \geq 0$: the **slope threshold**. If the elevations of the two cells differ by at most D, there is no slope effect.
- I: the **ignition threshold**. A cell catches fire if the total heat it receives in a step is at least I (see Ignition below).

Computing the heat of a cell

The pseudocode below computes the total heat received by the cell $t = (2, 2)$; every other cell is handled the same way. Wind and slope effects are applied to each burning neighbour separately: all multiplications are done first and the integer division last, and the results are then added up.

Algorithm 1: Total heat received by the cell $t = (2, 2)$

Input: Grid at the start of the step, parameters $H, (d_r, d_c), W, S, D$
Output: *heat*, the total heat received by $t = (2, 2)$

$heat \leftarrow 0$;
// Visit every burning cell in the 5×5 square around t
for $r_s \leftarrow 2 - 2$ **to** $2 + 2$ **do**
  for $c_s \leftarrow 2 - 2$ **to** $2 + 2$ **do**
    if (r_s, c_s) *is outside the grid* **then continue**;
    if $(r_s, c_s) = (2, 2)$ ***or*** (r_s, c_s) *is not burning* **then continue**;
    $a \leftarrow H$; $b \leftarrow 1$;
    $(o_r, o_c) \leftarrow (2 - r_s,\ 2 - c_s)$;
    // Wind effect
    if $(o_r, o_c) = (d_r, d_c)$ ***or*** $(o_r, o_c) = (2d_r, 2d_c)$ **then**
      $a \leftarrow a \cdot W$;
    else if $(o_r, o_c) = (-d_r, -d_c)$ ***or*** $(o_r, o_c) = (-2d_r, -2d_c)$ **then**
      $b \leftarrow b \cdot W$;
    // Slope effect
    $\Delta \leftarrow elevation(2, 2) - elevation(r_s, c_s)$;
    if $\Delta > D$ **then**
      $a \leftarrow a \cdot S$;
    else if $\Delta < -D$ **then**
      $b \leftarrow b \cdot S$;
    $heat \leftarrow heat + \lfloor a/b \rfloor$;
return *heat*

Ignition

A cell ignites (becomes *burning*) if all of the following hold:

- it is *unburnt*;
- its fuel is greater than 0;
- it is not a fire station;
- its total heat is at least I.

Fire stations do not block heat. They simply never ignite.

Burning

In every step, each cell that was burning at the start of the step loses 1 unit of fuel. When its fuel reaches 0, it becomes *burnt*.

All cells are updated at the same time: the heat is computed from the grid as it was at the start of the step, so a cell that ignites in a step only starts giving heat and losing fuel in the next step.

4 Firefighter Crews

The input gives a list of firefighter crews. Crews are numbered $0, 1, 2, \ldots$ in the order they appear in the input. Each crew has a **position** (a cell) and an amount of **water** w, with $0 \le w \le X$.

Parameters

The following integer parameters are given in the input:

- $R \geq 1$: the **movement radius**. In one step, a crew can walk up to R rows and up to R columns away from its current cell (see Walking below). It is the same for all crews.
- $X \geq 1$: the **tank capacity**, the most water a crew can carry. One unit of water puts out one burning cell, and a crew that ends a step on a fire station has its tank refilled to X. It is the same for all crews.

Walking

In one step, a crew can walk to any cell within R rows and R columns of its current cell. So the number of steps needed to walk from cell a to cell b is

$$s(a, b) = \left\lceil \frac{\max(|r_a - r_b|,\ |c_a - c_b|)}{R} \right\rceil .$$

When a crew walks towards a cell g, it moves $\min(R, |r_g - r_c|)$ rows and $\min(R, |c_g - c_c|)$ columns towards g, where (r_c, c_c) is its current cell. In particular, it reaches g in this step if $s(c, g) \leq 1$.

One step of a crew

The pseudocode below shows what one crew does in one step. Every crew follows it independently, reading the grid as it was at the start of the step. A crew does not remember its target: the target is chosen again from scratch every step. "Closest" means the smallest number of walking steps $s(c, t)$, with ties broken by the lowest row, then the lowest column. The same rule is used both for choosing a burning cell and for choosing a fire station. A crew puts out the cell it ends up on whether or not that cell was its target. If several crews with water end the step on the same burning cell, the cell becomes wet and each of those crews uses 1 unit of water. Crews never affect each other in any other way.

Algorithm 2: One step of a crew

Input: Grid G at the start of the step, the crew's cell c and water w, parameters R, X
Output: The crew's new cell c and water w

```
// Choose a target
if w > 0 then
    if no cell is burning in G then t ← none;
    else t ← the closest cell that is burning in G;
else
    t ← the closest fire station;
// Move
if t ≠ none then
    c ← the cell reached by walking towards t;
// Put out a fire or refill
if c is burning in G and w > 0 then
    mark c to become wet at the end of the step;
    w ← w − 1;
if c is a fire station then w ← X;
return (c, w)
```

5 Simulation Step Order

The simulation runs for T steps. Within one step, the fire and the crews both read the grid as it was at the start of the step (call it G), and their changes are combined at the end of the step:

1. **Fire:** compute the heat of every cell from G and ignite the eligible cells (Section 3). Every cell burning in G loses 1 unit of fuel and becomes burnt if its fuel reaches 0.
2. **Crews:** each crew chooses its target from G, makes its move, then puts out a fire or refills (Section 4). Whether a cell is burning is judged from G.
3. **Combine:** every cell put out by a crew becomes *wet*. It still loses 1 unit of fuel in this step, as in the Fire stage. This holds even if its fuel reached 0 in this step, so *wet* wins over *burnt*.

A cell put out in this step still gave heat in this step, because heat is computed from G.

The rules and pseudocode in this document are the authoritative definition of the simulation.

6 Input and Output Specifications

The compiled program is run as `./main <input file> <output file>`. The harness reads the input file, passes its contents to your `run_simulation()`, and writes the output file, so your code never reads or writes these files itself. `run_simulation()` receives the input as host arrays and must leave the final state grid, fuel grid and crews in the `state`, `fuel` and `crews` arrays, which the harness then writes out as described in Section 6.2. On entry, `state` is all unburnt, so your code must set the initial fires itself. Both files are binary. All integers are 32-bit signed little-endian (`int32`) unless stated otherwise, and all grids are stored row by row.

The script `convert.py` converts between the binary format and an equivalent text format (the same numbers in the same order, separated by whitespace, with comments after `#`), so that small tests can be written and read by hand:

- `python3 convert.py in2txt <input.bin> <input.txt>`
- `python3 convert.py in2bin <input.txt> <input.bin>`
- `python3 convert.py out2txt <input.bin> <output.bin> <output.txt>` (the output needs the input to know N and the number of crews)

Every public test case is given in both formats: the input as `<name>.in.bin` and `<name>.in.txt` in the `input` folder, and the expected output as `<name>.out.bin` and `<name>.out.txt` in the `output` folder.

6.1 Input Format

1. **Header:** 11 `int32` values, in this order: N, T, H, d_r, d_c, W, S, D, I, R, X.
2. **Fuel grid:** N^2 bytes (`uint8`).
3. **Elevation grid:** N^2 bytes (`uint8`).
4. **Initial fires:** an `int32` count F, then F pairs (row, column).
5. **Fire stations:** an `int32` count K, then K pairs (row, column).
6. **Crews:** an `int32` count C, then C triples (row, column, water).

6.2 Output Format

After simulating T steps, write:

1. **State grid:** N^2 bytes (`uint8`), with 0 = unburnt, 1 = burning, 2 = burnt, 3 = wet.
2. **Fuel grid:** N^2 bytes (`uint8`). Fire stations keep the fuel value given in the input.
3. **Crews:** for each crew in input order, a triple (row, column, water) of `int32`.

Fire stations always have state 0 (unburnt).

6.3 Your Own Test Cases

The public test cases cover only a few small scenarios, so you should also test your code on test cases of your own. To add one, put its input as `<name>.in.bin` in the `input` folder and its expected output as `<name>.out.bin` in the `output` folder; `run_testcases.sh` runs every input it finds there. Small inputs can be written by hand in the text format and converted with `convert.py in2bin`.

To get the expected outputs, consider first writing a short sequential version of the simulation (in C++ or Python) that follows the rules and pseudocode of Sections 3–5 directly, without worrying about speed, and writes its output in the format of Section 6.2. Run it on your inputs to produce the expected outputs, then compare your GPU code against it on many inputs, including large random ones.

7 Technical Constraints

Quantity	Limit
Grid size N	$1 \le N \le 8192$
Steps T	$1 \le T \le 1000$
Crews C	$0 \le C \le 1024$
Fire stations K	$0 \le K \le 256$, and $K \ge 1$ if $C \ge 1$
Initial fires F	$0 \le F \le N^2$
Base heat H	$1 \le H \le 1000$
Wind and slope factors W, S	$1 \le W, S \le 8$
Slope threshold D	$0 \le D \le 255$
Ignition threshold I	$1 \le I \le 10^9$
Movement radius R	$1 \le R \le 64$
Tank capacity X	$1 \le X \le 255$, and $0 \le$ each crew's initial water $\le X$

The input also guarantees that:

- no initial fire is on a fire station, and no list contains the same cell twice;
- every initial fire has fuel greater than 0;
- there is at least one fire station if there is at least one crew.

8 Parallel Implementation

The most direct way to parallelise the simulation is to launch one thread per cell in every step. Each thread reads the 5×5 square around its cell from G and computes the cell's new state and fuel. For the crews, each crew scans the whole grid for its closest burning cell.

This is correct, but most of the work it does is wasted. Only cells near the fire can change in a step: a cell can only ignite if some burning cell is within 2 rows and 2 columns of it, and only burning cells lose fuel. Early in the simulation the fire covers a tiny part of the grid, and later most of the area it has passed through is burnt and never changes again. Yet every step launches N^2 threads, over 67 million for $N = 8192$, so over $T = 1000$ steps the program performs about 6.7×10^{10} cell updates, almost all of which leave the cell unchanged. Likewise, scanning the whole grid costs N^2 work per crew per step, even when only a handful of cells are burning.

8.1 Worklist

Instead of visiting every cell in every step, your program can maintain a **worklist**: a list of only the cells that need attention. Here, the list of burning cells is enough. Every cell that can change in a step is either burning itself or within the 5×5 square of a burning cell. So in each step:

- one thread per burning cell can update its fuel and look at the cells in its 5×5 square that could ignite;
- the cells that are burning at the end of the step form the worklist for the next step.

The same list also gives the crews their candidate targets, so a crew only needs to look at the burning cells, not the whole grid.

A few things to think about:

- Several burning cells can share a neighbour, so the same unburnt cell may be found many times in one step. Its heat must still be computed from all of its burning neighbours, and it must be added to the next worklist only once.
- Many threads append to the same list at the same time, so each needs a way to reserve its own slot.
- The length of the list changes every step, so the number of threads to launch is not fixed.

8.2 Choosing Crew Targets

The direct way to choose targets is one thread per crew, with each thread looping over all burning cells to find its crew's closest one. This launches only C threads, at most 1024 and often far fewer, which is not enough to keep the GPU busy, and each of them does work proportional to the number of burning cells, one cell after another. With a few crews and a fire of millions of cells, this one kernel can take longer than the rest of the step.

Instead, the search for each crew can be split across many threads. For example, a block of threads can handle one crew: each thread finds the closest target among its share of the burning cells, and the block then combines these into a single answer with a parallel reduction. The combining step must respect the tie-break rule, so that equally close targets are resolved by the lowest row, then the lowest column.

8.3 Performance Tuning

A worklist is not always faster. Its cost grows with the number of burning cells, and each burning cell costs more than in the full-grid approach: threads read scattered cells, contend on atomic operations, and must avoid adding the same cell twice. The full-grid approach costs about the same in every step, however large the fire is. So when few cells are burning the worklist wins, and when the fire covers a large part of the grid a kernel over all cells may be faster.

Your program can use both: in each step, use the worklist if the number of burning cells is below some **threshold**, and the full grid otherwise. Find the best threshold experimentally. Time one step of each approach on grids with different numbers of burning cells, find where the two running times cross, and use that point as your threshold. Check whether the crossover depends on the grid size N; if it does, express the threshold as a fraction of N^2 rather than as a fixed count.

The same applies to choosing crew targets. When few cells are burning, one thread per crew is cheap and the reduction is pure overhead; when many are burning, splitting the search wins. Find this second threshold the same way, by timing both versions for different numbers of burning cells and crews.

9 Evaluation

The public test cases only check correctness. Your program is evaluated on a larger set of hidden test cases. For each test case, your output is compared byte by byte with the output of the reference implementation.

- **Correctness:** most tests exercise one part of the simulation at a time, with the other features turned off through the input (for example, no crews), and many run for a single step. A few tests run the full simulation.
- **Optimisations:** some large tests also have a time limit, and a test that exceeds it fails. These limits are set so that a correct program without the appropriate optimisations from Section 8 will exceed them.
- **Performance:** the running time of your program is compared with the average running time of all submissions. Depending on how far your time is from the average, you receive a bonus or a penalty of up to 2 marks.

10 Sample Execution Scenarios

A 6×6 grid, shown in the text format (comments after `#` are for explanation only). Every cell has fuel 5 and elevation 0, except cell $(1, 1)$, which has elevation 10. The wind blows east.

```
6 3 10 0 1 2 2 5 20 1 1      # N T H dr dc W S D I R X
5 5 5 5 5 5                  # fuel grid (6 rows)
5 5 5 5 5 5
5 5 5 5 5 5
5 5 5 5 5 5
5 5 5 5 5 5
5 5 5 5 5 5
0 0 0 0 0 0                  # elevation grid (6 rows)
0 10 0 0 0 0
0 0 0 0 0 0
0 0 0 0 0 0
0 0 0 0 0 0
0 0 0 0 0 0
1   2 1                      # 1 initial fire: (2,1)
1   0 5                      # 1 fire station: (0,5)
1   4 4 1                    # 1 crew at (4,4) with 1 unit of water
```

A single burning cell gives heat $H = 10$ to each cell around it, which is below $I = 20$. So on its own it ignites only the cells that get a bonus: downwind cells ($\times W = 2$) and uphill cells more than $D = 5$ higher ($\times S = 2$).

Below is the grid at the end of each step: `.` = unburnt, `F` = burning, `B` = burnt, `W` = wet, and `S` marks the station (whose state is unburnt). The fuel grid is shown next to the state grid.

```
After step 1                After step 2                After step 3
. . . . . S   5 5 5 5 5 5   F F F F F S   5 5 5 5 5 5   F F F F F S   4 4 4 4 4 5
. F . . . .   5 5 5 5 5 5   F F F F F .   5 4 5 5 5 5   F F F F F F   4 3 4 4 4 5
. F F F . .   5 4 5 5 5 5   . F W F F F   5 3 4 4 5 5   F F W F F F   5 2 4 3 4 4
. . . . . .   5 5 5 5 5 5   F F F F F .   5 5 5 5 5 5   F F F F F F   4 4 4 4 4 5
. . . . . .   5 5 5 5 5 5   F F F F F .   5 5 5 5 5 5   F F F F F F   4 4 4 4 4 5
. . . . . .   5 5 5 5 5 5   . . . . . .   5 5 5 5 5 5   F F F F F F   5 5 5 5 5 5
crew: (3,3), water 1        crew: (2,2), water 0        crew: (1,3), water 0
```

- **Step 1.** $(2, 1)$ loses 1 fuel. $(2, 2)$ and $(2, 3)$ are downwind of it and receive 20, and $(1, 1)$ is uphill and receives 20, so all three ignite. The crew's closest fire is $(2, 1)$, 3 steps away, so the crew walks to $(3, 3)$. Cells that ignite in this step are not burning in G, so they are not targets yet.
- **Step 2.** The four burning cells ignite many more cells. From $(3, 3)$, the closest burning cells are $(2, 2)$ and $(2, 3)$, both 1 step away. The tie goes to the lower column, so the crew walks to $(2, 2)$ and puts it out. The cell becomes wet and the crew has 0 water left.
- **Step 3.** The wet cell $(2, 2)$ no longer burns, so it keeps its fuel of 4. The crew has no water, so it heads for the fire station $(0, 5)$ and walks to $(1, 3)$.

The output file contains the final state grid, the final fuel grid, and the crew `1 3 0` (row 1, column 3, water 0).